Fibonacci 예제 분석 (Fibonacci Example Analysis)

개요

이 문서에서는 Plonky2의 `fibonacci.rs` 예제를 라인별로 상세히 분석합니다. 이 예제는 "나는 초기값 a와 b로 시작하는 피보나치 수열의 100번째 원소를 알고 있다"는 명제를 증명하는 방법을 보여줍니다. a == 0, b == 1일 때는 표준 피보나치 수열의 100번째 수를 증명하게 됩니다.

이 예제는 Plonky2의 기본적인 사용 패턴을 보여주는 가장 간단한 예제로, 다음 개념들을 학습할 수 있습니다:

- CircuitBuilder를 사용한 회로 구성
- 가상 타겟(Virtual Target) 추가
- 산술 연산 추가
- 공개 입력(Public Input) 등록
- Witness 설정
- 증명 생성 및 검증

전체 코드 구조

```rust
use anyhow::Result;

use plonky2::field::types::Field;

use plonky2::iop::witness::{PartialWitness, WitnessWrite};

use plonky2::plonk::circuit_builder::CircuitBuilder;
```

```rust
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};

fn main() -> Result<()> {
// 1. 타입 설정
const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;

// 2. CircuitBuilder 생성
let config = CircuitConfig::standard_recursion_config();
let mut builder = CircuitBuilder::<F, D>::new(config);

// 3. 회로 구성
let initial_a = builder.add_virtual_target();
let initial_b = builder.add_virtual_target();
let mut prev_target = initial_a;
let mut cur_target = initial_b;
for _ in 0..99 {
let temp = builder.add(prev_target, cur_target);
prev_target = cur_target;
cur_target = temp;
}
```

```rust
// 4. 공개 입력 등록
builder.register_public_input(initial_a);
builder.register_public_input(initial_b);
builder.register_public_input(cur_target);

// 5. Witness 설정
let mut pw = PartialWitness::new();
pw.set_target(initial_a, F::ZERO)?;
pw.set_target(initial_b, F::ONE)?;

// 6. 회로 빌드 및 증명 생성
let data = builder.build::<C>();
let proof = data.prove(pw)?;

// 7. 결과 출력 및 검증
println!(
"100th Fibonacci number mod |F| (starting with {}, {}) is: {}",
proof.public_inputs[0], proof.public_inputs[1], proof.public_inputs[2]
);

data.verify(proof)
```

```
}
```

라인별 상세 분석

1. 의존성 임포트

```rust
use anyhow::Result;
use plonky2::field::types::Field;
use plonky2::iop::witness::{PartialWitness, WitnessWrite};
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};
```

설명:

- `anyhow::Result` : 에러 처리를 위한 타입
- `Field` : 유한체 연산을 위한 트레이트
- `PartialWitness` : 회로의 비공개 입력값을 저장하는 구조체
- `WitnessWrite` : Witness에 값을 설정하기 위한 트레이트
- `CircuitBuilder` : 회로를 구성하기 위한 빌더 패턴 구조체
- `CircuitConfig` : 회로의 보안 수준 및 설정을 정의하는 구조체
- `GenericConfig` : SNARK 설정을 위한 제네릭 트레이트
- `PoseidonGoldilocksConfig` : Poseidon 해시와 Goldilocks Field를 사용하는 구체적인 설정

2. 타입 설정

```
const D: usize = 2;

type C = PoseidonGoldilocksConfig;

type F = <C as GenericConfig<D>>::F;
```

설명:

`const D: usize = 2`

- `D` 는 확장 차수(Extension Degree)를 나타냅니다
- `D = 2` 는 2차 확장체를 사용한다는 의미입니다
- Plonky2는 기본 필드 F 위에 F[X]/(X^2 - 7)의 2차 확장체를 사용합니다
- 이는 FRI 프로토콜에서 더 강력한 보안을 제공합니다

`type C = PoseidonGoldilocksConfig`

- `C` 는 SNARK 시스템의 전체 설정을 나타내는 타입 별칭입니다
- `PoseidonGoldilocksConfig` 는 다음을 포함합니다:
- **Field**: Goldilocks Field (p = 2^64 - 2^32 + 1)
- **Hasher**: Poseidon 해시 함수
- **InnerHasher**: 내부 해시 함수 (Poseidon)
- Poseidon은 SNARK 친화적인 해시 함수로, 회로 내에서 효율적으로 계산 가능합니다

`type F = <C as GenericConfig<D>>::F`

- `F` 는 기본 필드(Base Field) 타입을 나타냅니다
- 이 경우 Goldilocks Field가 됩니다
- 연관 타입(Associated Type) 문법을 사용하여 Config에서 Field 타입을 추출합니다

3. CircuitBuilder 생성

```
let  config = CircuitConfig::standard_recursion_config();

let  mut  builder = CircuitBuilder::<F, D>::new(config);
```

설명:

`CircuitConfig::standard_recursion_config()`

- 재귀 증명에 적합한 표준 설정을 생성합니다
- 주요 설정값:
- `security_bits` : 100 (보안 수준)
- `num_wires` : 135 (와이어 수)
- `num_routed_wires` : 80 (라우팅 가능한 와이어 수)
- `num_constants` : 2 (상수 게이트당 상수 개수)
- `use_base_arithmetic_gate` : true (기본 산술 게이트 사용)
- `fri_config` : FRI 프로토콜 설정
- `rate_bits` : 3 (코드 레이트 = 1/8)
- `cap_height` : 4 (Merkle 트리 캡 높이)
- `num_query_rounds` : 28 (쿼리 라운드 수)

`CircuitBuilder::<F, D>::new(config)`

- 주어진 설정으로 새로운 `CircuitBuilder` 인스턴스를 생성합니다
- 내부적으로 다음을 초기화합니다:
- 빈 게이트 집합
- 빈 제약조건 리스트
- 빈 공개 입력 리스트
- Witness 생성기 리스트

- `check_config()` 를 호출하여 설정의 일관성을 검증합니다

4. 초기값 Target 추가

```
let initial_a = builder.add_virtual_target();
let initial_b = builder.add_virtual_target();
```

설명:

`add_virtual_target()`

- 가상 타겟(Virtual Target)을 하나 추가합니다
- 가상 타겟은 실제 와이어가 아닌 추상적인 변수를 나타냅니다
- 내부 동작:

```
pub fn add_virtual_target(&mut self) -> Target {
let index = self.virtual_target_index;
self.virtual_target_index += 1;
Target::VirtualTarget { index }
}
```

- 반환되는 `Target` 은 나중에 실제 와이어로 매핑됩니다

이 예제에서의 역할:

- `initial_a` : 피보나치 수열의 첫 번째 값 (F_0)
- `initial_b` : 피보나치 수열의 두 번째 값 (F_1)

5. Fibonacci 계산 회로 구성

```
let mut prev_target = initial_a;

let mut cur_target = initial_b;

for _ in 0..99 {

let temp = builder.add(prev_target, cur_target);

prev_target = cur_target;

cur_target = temp;

}
```

설명:

이 부분은 피보나치 수열을 계산하는 회로를 구성합니다. 99번의 반복을 통해 F_2부터 F_100까지 계산합니다.

반복 과정:

- **초기 상태**: `prev_target = F_0`, `cur_target = F_1`
- **1회 반복**: `temp = F_0 + F_1 = F_2`, `prev_target = F_1`, `cur_target = F_2`
- **2회 반복**: `temp = F_1 + F_2 = F_3`, `prev_target = F_2`, `cur_target = F_3`
- ...
- **99회 반복**: `temp = F_98 + F_99 = F_100`, `prev_target = F_99`, `cur_target = F_100`

`builder.add(prev_target, cur_target)`

- 두 타겟의 덧셈 연산을 회로에 추가합니다
- 내부 동작:

```
pub fn add(&mut self, a: Target, b: Target) -> Target {

if let (Target::Constant(ca), Target::Constant(cb)) = (a, b) {
```

```
// 두 상수의 덧셈은 컴파일 타임에 계산
return  self.constant(ca + cb);
}
// ArithmeticGate를 사용하여 덧셈 연산 추가
let  gate = ArithmeticGate::new_from_config(&self.config);
let (gate_index, wire_index) = self.add_gate(gate, vec![]);
// 입력 와이어 연결
self.connect(a, Target::wire(gate_index, gate.wire_ith_multiplicand_0(wire_index
self.connect(b, Target::wire(gate_index, gate.wire_ith_multiplicand_1(wire_index
// 출력 와이어 반환
Target::wire(gate_index, gate.wire_ith_output(wire_index))
}
```

생성되는 제약조건:

- 각 덧셈 연산마다 `temp = prev_target + cur_target` 제약조건이 추가됩니다
- 이는 ArithmeticGate를 통해 구현됩니다
- 총 99개의 덧셈 제약조건이 생성됩니다

6. 공개 입력 등록

```
builder.register_public_input(initial_a);
builder.register_public_input(initial_b);
builder.register_public_input(cur_target);
```

설명:

공개 입력(Public Input)은 증명과 함께 공개되는 값들입니다. 검증자는 이 값들을 알고 있으며, 증명이 이 값들에 대해 올바른지 확인합니다.

`register_public_input(target: Target)`

- 주어진 타겟을 공개 입력으로 등록합니다
- 내부 동작:

```
pub fn register_public_input(&mut self, target: Target) {
self.public_inputs.push(target);
}
```

- 공개 입력은 순서대로 저장되며, 증명의 `public_inputs` 필드에 포함됩니다

이 예제에서의 공개 입력:

1. `initial_a` (F_0): 피보나치 수열의 첫 번째 값
2. `initial_b` (F_1): 피보나치 수열의 두 번째 값
3. `cur_target` (F_100): 계산된 100번째 피보나치 수

의미:

- 증명자는 "나는 F_0 = 0, F_1 = 1로 시작하여 F_100을 올바르게 계산했다"는 것을 증명합니다
- 검증자는 F_0, F_1, F_100 값을 알고 있으며, 증명이 이 값들에 대해 올바른지 확인합니다
- 실제 계산 과정(중간 값들)은 비공개로 유지됩니다

7. Witness 설정

```
let mut pw = PartialWitness::new();
pw.set_target(initial_a, F::ZERO)?;
```

```
pw.set_target(initial_b, F::ONE)?;
```

설명:

Witness는 회로의 모든 와이어에 대한 구체적인 값들을 포함합니다. 증명자만 알고 있는 비공개 정보입니다.

`PartialWitness::new()`

- 새로운 부분 Witness를 생성합니다
- "부분(Partial)"이라는 이름은 모든 와이어 값을 직접 설정하지 않아도 되기 때문입니다
- Witness 생성기(Generator)가 나머지 값들을 자동으로 계산합니다

`pw.set_target(target, value)`

- 특정 타겟에 구체적인 값을 설정합니다
- 내부 동작:

```
fn set_target(&mut self, target: Target, value: F) -> Result<()> {
match target {
Target::VirtualTarget { index } => {
if let Some(old_value) = self.target_values.insert(index, value) {
ensure!(old_value == value, "Conflicting values for target");
}
Ok(())
}
Target::Wire(wire) => {
// Wire 타겟의 경우 직접 설정
self.set_wire(wire, value)
```

```
}
}
}
```

이 예제에서의 설정:

- `initial_a = F::ZERO = 0` : 첫 번째 피보나치 수
- `initial_b = F::ONE = 1` : 두 번째 피보나치 수
- 나머지 중간 값들(F_2, F_3, ..., F_100)은 자동으로 계산됩니다

Witness 생성 과정:

1. 사용자가 초기값만 설정 (`initial_a` , `initial_b`)
2. `generate_partial_witness()` 함수가 호출됨
3. 각 게이트의 제약조건을 만족하도록 나머지 와이어 값들을 계산
4. 이 예제에서는 각 덧셈 게이트가 순차적으로 실행되어 F_2부터 F_100까지 계산됨

8. 회로 빌드

```
let data = builder.build::<C>();
```

설명:

`build()` 함수는 `CircuitBuilder` 를 소비하고 `CircuitData` 를 생성합니다. 이는 증명 생성과 검증에 필요한 모든 정보를 포함합니다.

내부 동작:

```
pub fn build<C: GenericConfig<D, F = F>>(self) -> CircuitData<F, C, D> {
// 1. 회로 크기 계산
```

```
let degree_bits = self.compute_degree_bits();

let degree = 1 << degree_bits;

// 2. 게이트 배치 및 와이어 라우팅

let gates = self.gate_instances;

// 3. Copy Constraint 처리 (Permutation Argument)

let (sigmas, copy_constraints) = self.process_copy_constraints();

// 4. Witness 생성기 정렬

let generators = self.generators;

// 5. FFT 준비

let fft_root_table = self.compute_fft_root_table(degree);

// 6. CircuitData 생성

CircuitData {

prover_only: ProverOnlyCircuitData { ... },

verifier_only: VerifierOnlyCircuitData { ... },

common: CommonCircuitData { ... },

}

}
```

생성되는 데이터 구조:

`ProverOnlyCircuitData` (증명자만 필요한 데이터):

- `generators` : Witness 생성기 리스트
- `sigmas` : Permutation 다항식 (Copy Constraint 처리)
- `subgroup` : FFT 서브그룹

- `constants_sigmas_commitment` : 상수 및 시그마 커밋먼트

`VerifierOnlyCircuitData` (검증자만 필요한 데이터):

- `constants_sigmas_cap` : 상수 및 시그마 Merkle 트리 캡
- `circuit_digest` : 회로의 해시 (Fiat-Shamir에 사용)

`CommonCircuitData` (공통 데이터):

- `config` : 회로 설정
- `fri_params` : FRI 프로토콜 파라미터
- `gates` : 사용된 게이트 타입들
- `degree_bits` : 회로 크기 (log2)
- `quotient_degree_factor` : 몫 다항식 차수 계수

9. 증명 생성

```
let proof = data.prove(pw)?;
```

설명:

`prove()` 함수는 주어진 Witness로부터 영지식증명을 생성합니다. 이는 Plonky2의 핵심 기능입니다.

내부 동작 (간략):

```
pub fn prove(&self, inputs: PartialWitness<F>) -> Result<ProofWithPublicInputs

// 1. 전체 Witness 생성

let witness = generate_partial_witness(inputs, &self.prover_only, &self.common)

// 2. Wire 값들을 다항식으로 변환

let wires_commitment = commit_wires(&witness);
```

```
// 3. Fiat-Shamir 챌린지 생성
let (betas, gammas) = generate_challenges(&wires_commitment);
// 4. Permutation Argument (Z 다항식)
let  zs_commitment = compute_permutation_zs(&witness, &betas, &gammas);
// 5. Quotient Polynomial 계산
let  quotient_commitment = compute_quotient_polys(&witness, &challenges);
// 6. Opening Proof (FRI)
let  opening_proof = prove_openings(&commitments, &challenges);
// 7. 증명 구성
Proof {
wires_cap,
plonk_zs_partial_products_cap,
quotient_polys_cap,
openings,
opening_proof,
}
}
```

증명 생성 단계:

1. **Witness 생성**: 초기값으로부터 모든 와이어 값 계산
2. **Wire Commitment**: 와이어 값들을 다항식으로 변환하고 Merkle 트리로 커밋
3. **Permutation Argument**: Copy Constraint를 만족하는 Z 다항식 생성
4. **Quotient Polynomial**: 모든 제약조건을 하나의 다항식으로 결합

5. **FRI Proof**: 다항식이 낮은 차수임을 증명

자세한 내용은 03-proof-generation.md를 참조하세요.

10. 결과 출력

```
println!(
"100th Fibonacci number mod |F| (starting with {}, {}) is: {}",
proof.public_inputs[0], proof.public_inputs[1], proof.public_inputs[2]
);
```

설명:

증명 객체의 `public_inputs` 필드에는 등록된 공개 입력 값들이 순서대로 저장되어 있습니다.

출력 예시:

```
100th Fibonacci number mod |F| (starting with 0, 1) is: 3736710778780434371
```

주의사항:

- 결과는 Goldilocks Field의 원소이므로 `mod (2^64 - 2^32 + 1)` 연산이 적용됩니다
- 실제 100번째 피보나치 수는 매우 큰 수이지만, 필드 연산으로 인해 모듈로 값이 출력됩니다
- 표준 피보나치 수열의 100번째 수: F_100 = 354,224,848,179,261,915,075

11. 증명 검증

```
data.verify(proof)
```

설명:

`verify()` 함수는 생성된 증명이 올바른지 검증합니다.

내부 동작 (간략):

```
pub fn verify(&self, proof: ProofWithPublicInputs<F, C, D>) -> Result<()> {
// 1. 공개 입력 해시 검증
let public_inputs_hash = hash_public_inputs(&proof.public_inputs);
// 2. Fiat-Shamir 챌린지 재생성
let challenges = get_challenges(&proof, &public_inputs_hash, &self.verifier_onl
// 3. Vanishing Polynomial 평가
let vanishing_polys_zeta = eval_vanishing_poly(&proof.openings, &challenges);
// 4. Quotient 검증
verify_quotient(&vanishing_polys_zeta, &proof.openings, &challenges)?;
// 5. FRI 검증
verify_fri_proof(&proof.opening_proof, &challenges)?;
Ok(())
}
```

검증 단계:

1. **공개 입력 검증**: 공개 입력이 올바르게 해시되었는지 확인
2. **챌린지 재생성**: 증명자와 동일한 챌린지를 Fiat-Shamir로 재생성
3. **제약조건 검증**: 모든 게이트 제약조건과 Copy Constraint가 만족되는지 확인
4. **FRI 검증**: 커밋된 다항식이 실제로 낮은 차수인지 확인

자세한 내용은 [04-proof-verification.md]를 참조하세요.

생성되는 Gate 분석

ArithmeticGate

이 예제에서는 주로 `ArithmeticGate` 가 사용됩니다. `ArithmeticGate` 는 다음과 같은 제약조건을 표현합니다:

```
const_0 * multiplicand_0 * multiplicand_1 + const_1 * addend = output
```

Fibonacci 덧셈의 경우:

- `const_0 = 0` (곱셈 항 비활성화)
- `const_1 = 1` (덧셈 항 활성화)
- `addend = prev_target + cur_target`
- `output = temp`

실제로는 다음과 같이 단순화됩니다:

```
prev_target + cur_target = temp
```

Gate 수 계산

예상 Gate 수:

- **덧셈 Gate**: 99개 (각 반복마다 하나씩)
- **공개 입력 Gate**: 필요에 따라 추가
- **상수 Gate**: 초기값 설정을 위해 필요할 수 있음

실제 Gate 수는 다음 요인에 따라 달라집니다:

- 회로 최적화

- 와이어 라우팅
- 패딩 (2의 거듭제곱으로 맞추기 위해)

Wire 연결 구조

```
초기값:

initial_a (F_0) ──┐
                  ├──> [Add Gate 1] ──> temp_1 (F_2)
initial_b (F_1) ──┘ |
                    |
initial_b (F_1) ──┐ |
                  ├──> [Add Gate 2] ──> temp_2 (F_3)
temp_1 (F_2) ─────┘ |
                    |
temp_1 (F_2) ─────┐ |
                  ├──> [Add Gate 3] ──> temp_3 (F_4)
temp_2 (F_3) ─────┘ |
                    |
                ... |
                    |
temp_97 (F_98) ───┐ |
                  ├──> [Add Gate 99] ──> cur_target (F_100)
temp_98 (F_99) ───┘
```

```
공개 입력:

initial_a (F_0)

initial_b (F_1)

cur_target (F_100)
```

Copy Constraint:

- `initial_b` 는 첫 번째와 두 번째 덧셈 게이트에서 사용되므로 Copy Constraint가 필요합니다
- 각 중간 결과(`temp_i`)는 다음 덧셈의 입력으로 사용되므로 Copy Constraint로 연결됩니다
- 총 약 100개 이상의 Copy Constraint가 생성됩니다

실행 결과 및 증명 크기

실행 방법

```
cd plonky2/plonky2

cargo run --release --example fibonacci
```

예상 출력

```
100th Fibonacci number mod |F| (starting with 0, 1) is: 3736710778780434371

Proof generated successfully

Verification successful
```

증명 크기 분석

Plonky2의 증명 크기는 매우 작습니다. 일반적인 구성에서:

증명 구성 요소:

1. **Merkle Caps**: 각 커밋먼트마다 하나씩

- Wires Cap: ~256 bytes
- Z's Cap: ~256 bytes
- Quotient Cap: ~256 bytes

2. **Openings**: 평가 지점에서의 다항식 값들

- 각 Opening: 8 bytes (Goldilocks Field 원소)
- 약 20-30개의 Opening: ~200 bytes

3. **FRI Proof**:

- Commit Phase: 각 라운드마다 Merkle Cap (~256 bytes × 라운드 수)
- Query Phase: 각 쿼리마다 Merkle Proof (~1 KB × 쿼리 수)
- Final Polynomial: ~100 bytes

총 증명 크기: 약 45-100 KB

이는 회로 크기에 거의 무관하게 일정합니다 (로그 스케일링).

성능 특성

증명 생성 시간:

- 이 간단한 예제: ~10-50ms (릴리스 빌드)
- 주요 시간 소요:
- FFT 연산: ~40%
- Merkle 트리 구성: ~30%
- 다항식 연산: ~20%

- 기타: ~10%

검증 시간:

- ~5-10ms (릴리스 빌드)
- 회로 크기에 거의 무관 (로그 스케일링)

메모리 사용:

- 증명 생성: ~100-500 MB
- 검증: ~10-50 MB

학습 포인트

이 예제를 통해 다음을 학습할 수 있습니다:

1. **CircuitBuilder 기본 사용법**

- 타겟 추가 (`add_virtual_target`)
- 연산 추가 (`add`)
- 공개 입력 등록 (`register_public_input`)

2. **Witness 설정**

- `PartialWitness` 사용법
- 초기값만 설정하고 나머지는 자동 계산

3. **증명 생성 및 검증 흐름**

- `build()` → `prove()` → `verify()`
- 각 단계의 역할과 의미

4. **회로 설계 패턴**

- 반복적인 계산을 회로로 표현하는 방법
- 중간 결과를 타겟으로 관리하는 방법

5. **공개 입력의 역할**

- 무엇을 공개하고 무엇을 비공개로 할지 결정
- 이 예제에서는 초기값과 최종 결과만 공개

다음 단계

더 복잡한 예제를 학습하려면 다음 문서를 참조하세요:

- 08-example-range-check.md: Range Check와 Lookup Table 사용법
- 09-example-square-root.md: Custom Generator 작성법
- 10-recursive-proofs.md: 재귀 증명 구성법

참고 자료

- 02-circuit-building.md: CircuitBuilder 상세 설명
- 03-proof-generation.md: 증명 생성 프로세스
- 04-proof-verification.md: 증명 검증 프로세스
- Plonky2 GitHub: https://github.com/0xPolygonZero/plonky2

문서 네비게이션

- **이전**: 06. Goldilocks Field - 필드 연산 구현
- **다음**: 08. Range Check 예제 - Lookup 사용 예제
- **관련 문서**:
- 02. 회로 구성 프로세스 - CircuitBuilder 상세

- 03. 증명 생성 프로세스 - 증명 생성 과정
- 04. 증명 검증 프로세스 - 증명 검증 과정
- 08. Range Check 예제 - 다음 예제
- 09. Square Root 예제 - 고급 예제